NLP Lab GPU Productivity Guide

Access

1. Apply for a CS account here if you do not have one. You should select Prof. Kathleen McKeown or Prof. Smaranda Muresan as your account sponsor, depending on who you are working with.
2. If you are an undergraduate/MS student, ask the PhD student you are working with to email crf@cs.columbia.edu and grant machine access to your CS account. If you are a PhD student, directly email crf@cs.columbia.edu for machine access. Make sure you CC the faculty advisor when emailing so that they can review and approve your requests.
3. After you are approved, use ssh your_cs_account_name@hostname.cs.columbia.edu to log into the machines. Authenticate with your CS account password.

Machine Status

I would recommend using one of the following machines:

- seahorse.cs.columbia.edu: 8 * NVIDIA RTX A6000 48 GB, no NVLink
- piranha.cs.columbia.edu: 4 * NVIDIA A100 40 GB, with NVLink
- tigerfish.cs.columbia.edu: 4 * NVIDIA A100 40 GB, with NVLink
- kingcrab.cs.columbia.edu: 4 * NVIDIA V100 32 GB, with NVLink

Best Practices

Store everything on local SSDs.

It is **highly recommended** that you create project-specific folders on **local SSDs** and store all your files there, if possible. You can do this by creating a folder under **/local/nlp/** and then store everything in this folder. More specifically, make sure to:

- Create project-specific virtual environments in this folder so that your Python interpreter loads the Python libraries from the SSDs as well. This would also help you manage the environment dependencies.
- Make sure PyTorch and Hugging Face libraries save and load the model weights from the local SSD folder. Here is how to change the default cache folder for PyTorch and Hugging Face.
- Put all your code and data inside in this folder. Save your model and results there, too.

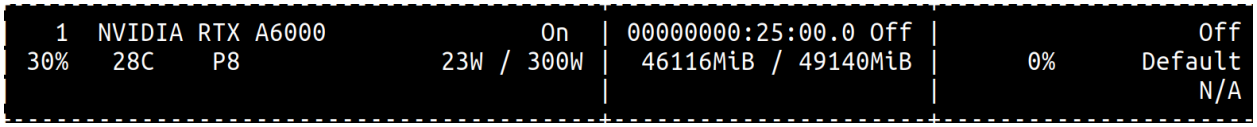
Files on local SSDs are not synchronized across multiple machines unfortunately.

Local SSDs are limited resources that are shared across lab members. Please delete unnecessary files (including the PyTorch and Hugging Face cache) as soon as possible so you are not monopolizing the storage space. It's always a good idea to monitor the overall disk space with df -h and your own space usage with du -sh /path/to/folder. Make sure you clean up the PyTorch and Hugging Face cache too as the model weights can be huge.

The rationale for this suggestion is that the home folder and the shared storage (/mnt/nlp_swordfish) are using network attached storage (NAS) backed by HDDs, which is connected to our servers using a 100 MBps link. Therefore, you will likely see 10-100x better disk IO performance when using the local SSDs. We are working with the CS department on upgrading the shared storage, and will update the guide again when that is done.

Do not leave idle programs on GPUs

We often see students leave programs idle on GPUs, as shown in the following nvidia-smi command report:



Here, the GPU is not doing any work, as demonstrated by the 0% utilization rate. However, the idling program still occupies a significant amount of memory, as 46 GB out of the 48GB is marked as used, which prevents other students from using this GPU.

We typically see this happen with Jupyter Notebooks that are open but not running any actual code. It's OK to keep the Jupyter Notebooks open if you are working on the code and experiments, but please do not let this happen over extended periods of time. Always terminate your Jupyter processes after you are done with the experiments.

Specify GPUs to use for your experiments.

Use nvidia-smi to check the status of GPUs on your machine, and then use export CUDA_VISIBLE_DEVICES=id_0,id_1,...,id_n to specify the GPUs you want to use. More details could be found here.

Do not enable DeepSpeed or PyTorch FSDP on machines without NVLink

Basically what the title says. For machines without NVLink (e.g. seahorse), it would take a significant amount of time to synchronize data between GPUs, and multi-GPU training would be slower than using a single GPU if DeepSpeed or FSDP is enabled. Enabling PyTorch DDP

might still be OK on these machines, especially if the number of trainable parameters is moderate.

Use the highest batch size possible, especially for local LLM inference

A lot of training tasks, and almost all of the LLM inference tasks, are bound by the GPU memory bandwidth. Using a larger batch size could significantly improve your training/inference throughput. For example, in LLM inference, changing the batch size from 1 to 8 would make the inference 8x faster, as the task is entirely bound by GPU memory bandwidth.

Use TF32, Mixed Precision, Flash Attention

Enabling TF32 can usually give you a 2-3x speed up with no loss in task accuracy. It's also super easy to enable by setting a few flags in PyTorch.

Using mixed precision (e.g. BF16/FP16) can give you a further 2-3x speed up, typically with no adverse effects on task accuracy.

If you are using a Transformer based model, consider enabling Flash Attention. This could give you huge memory savings, allowing you to use larger batch sizes. Flash Attention could also give you 1.2-1.6x speed up depending on the configuration. Follow this guide to install Flash Attention in your environment, and this documentation for enabling it in HF Transformers.